DAA
PR-1
Write a program to calculate Fibonacci numbers and find its step count

```
# Iterative Approach
def fibonacci_iterative(n):
    a = 0
    b = 1
    step_count = 0
    print("Iterative Fibonacci Series:")
    for i in range(n):
        print(a, end=" ")
        c = a + b
        a = b
        b = c
        step_count += 1
    print("\nIterative Step Count:", step_count)

# Recursive Approach
def fibonacci_recursive(n):
    global recursive_count
    recursive_count += 1

    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)

# Main Program
n = int(input("Enter the number of terms: "))
fibonacci_iterative(n)
recursive_count = 0
print("\nRecursive Fibonacci Series:")
for i in range(n):
    print(fibonacci_recursive(i), end=" ")
print("\nRecursive Step Count:", recursive_count)
```

1. Iterative Approach
The iterative approach calculates Fibonacci numbers using a for loop. It initializes two variables (a = 0, b = 1) and calculates the next number by adding the previous two numbers. The variables are updated in each iteration until the required number of terms is generated.
2. Recursive Approach
The recursive approach calculates Fibonacci numbers by calling the same function repeatedly. It uses the formula $F(n)=F(n-1)+F(n-2)$. The recursion stops when the base conditions n == 0 or n == 1 are reached.

Complexity comparison:

Parameter	Iterative	Recursive
Time Complexity	O(n)	$O(2^n)$
Space Complexity	O(1)	O(n)

Algorithm: Fibonacci Series Using Iterative Approach
1. Start.
2. Define a function fibonacci_iterative(n).

3. Initialize a = 0, b = 1, and step_count = 0.
4. Repeat the loop from 0 to n-1.
5. Display the current value of a.
6. Calculate the next Fibonacci number: c = a + b.
7. Update a = b and b = c.
8. Increment the step count.
9. Display the total step count.
10. Stop.

PR-2
Write a program to implement Huffman Encoding using a greedy strategy. / Implement job sequencing with deadlines using a greedy method.

```
import heapq

# ------------------------------
# A. HUFFMAN ENCODING
# ------------------------------

class Node:
    def __init__(self, char, freq):
        self.char = char
        self.freq = freq
        self.left = None
        self.right = None

    def __lt__(self, other):
        return self.freq < other.freq


def huffman_encoding(chars, freq):
    heap = []

    for char, frequency in zip(chars, freq):
        heapq.heappush(heap, Node(char, frequency))

    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)

        new_node = Node(None, left.freq + right.freq)
        new_node.left = left
        new_node.right = right

        heapq.heappush(heap, new_node)

    root = heap[0]

    def generate_codes(node, code):
        if node.char is not None:
            print(node.char, ":", code if code else "0")
            return

        generate_codes(node.left, code + "0")
```

```
        generate_codes(node.right, code + "1")

    print("\nHuffman Codes:")
    generate_codes(root, "")


# ------------------------------
# B. JOB SEQUENCING
# ------------------------------

def job_sequencing(jobs):
    jobs.sort(key=lambda x: x[2], reverse=True)

    max_deadline = max(job[1] for job in jobs)
    slots = [None] * max_deadline

    total_profit = 0

    for job_id, deadline, profit in jobs:
        for j in range(min(deadline, max_deadline) - 1, -1, -1):
            if slots[j] is None:
                slots[j] = job_id
                total_profit += profit
                break

    print("\nScheduled Jobs:")

    for i, job in enumerate(slots):
        if job is not None:
            print("Time Slot", i + 1, ":", job)

    print("Maximum Profit:", total_profit)


# ------------------------------
# MAIN PROGRAM
# ------------------------------

while True:
    print("\n========== DAA GREEDY ALGORITHMS ==========")
    print("1. Huffman Encoding")
    print("2. Job Sequencing with Deadlines")
    print("3. Exit")

    choice = int(input("\nEnter your choice: "))

    if choice == 1:
        n = int(input("Enter number of characters: "))

        chars = []
        freq = []

        for i in range(n):
            char = input("Enter character: ")
```

```
            frequency = int(input("Enter frequency: "))

            chars.append(char)
            freq.append(frequency)

        huffman_encoding(chars, freq)

    elif choice == 2:
        n = int(input("Enter number of jobs: "))

        jobs = []

        for i in range(n):
            job_id = input("Enter Job ID: ")
            deadline = int(input("Enter Deadline: "))
            profit = int(input("Enter Profit: "))

            jobs.append((job_id, deadline, profit))

        if jobs:
            job_sequencing(jobs)
        else:
            print("No jobs entered.")

    elif choice == 3:
        print("Exiting Program...")
        break

    else:
        print("Invalid Choice! Try Again.")
```

1. Huffman Encoding
Huffman Encoding is a lossless data compression technique that uses a greedy strategy. It assigns variable-length binary codes to characters based on their frequencies. Characters with higher frequencies receive shorter codes, while characters with lower frequencies receive longer codes.
2. Job Sequencing with Deadlines
Job Sequencing is a greedy algorithm used to maximize total profit by scheduling jobs before their deadlines. Each job has a deadline and a profit, and requires one unit of time. Jobs are selected in descending order of profit and scheduled in the latest available time slot before their deadline.

1. Huffman Encoding
Algorithm:
1. Start.
2. Accept characters and their frequencies.
3. Create a leaf node for each character.
4. Insert all nodes into a min-heap.
5. Extract the two nodes with minimum frequencies.
6. Combine them into a new node by adding their frequencies.
7. Insert the new node into the min-heap.
8. Repeat until only one node remains.
9. Traverse the Huffman tree and assign 0 to the left branch and 1 to the right branch.
10. Display the Huffman codes.
11. Stop.

Complexity:
- Time Complexity: $O(n\log n)$
- Space Complexity: $O(n)$

2. Job Sequencing with Deadlines
Algorithm:
1. Start.
2. Accept job ID, deadline and profit.
3. Sort jobs in descending order of profit.
4. Find the maximum deadline.
5. Create empty time slots.
6. Select each job according to its profit.
7. Check for an available slot from its deadline backwards.
8. If a slot is available, assign the job to it.
9. Calculate and display the total profit.
10. Stop.
Complexity:
- Time Complexity: $O(n\log n+nd)$
- Space Complexity: $O(d)$.

PR-3
Write a program to solve a fractional Knapsack problem using a greedy method.

```
# Fractional Knapsack Problem using Greedy Method
def fractional_knapsack(capacity, items):
    # Sort items by decreasing profit/weight ratio
    items.sort(key=lambda x: x[1] / x[0], reverse=True)

    total_profit = 0
    print("\nSelected Items:")
    for weight, profit in items:
        if capacity == 0:
            break
        if weight <= capacity:
            capacity -= weight
            total_profit += profit
            print("Weight:", weight, "Profit:", profit, "Fraction: 1")
        else:
            fraction = capacity / weight
            total_profit += profit * fraction
            print("Weight:", weight, "Profit:", profit,
                  "Fraction:", round(fraction, 2))
            capacity = 0
    print("\nMaximum Profit:", round(total_profit, 2))


# Main Program

n = int(input("Enter number of items: "))
items = []
for i in range(n):
    weight = float(input("Enter weight of item: "))
    profit = float(input("Enter profit of item: "))
```

```
    items.append((weight, profit))
capacity = float(input("Enter knapsack capacity: "))
fractional_knapsack(capacity, items)
```

The Fractional Knapsack problem is a greedy algorithm used to maximize profit within a given weight capacity. Items are selected based on their profit-to-weight ratio. Unlike the 0/1 Knapsack problem, items can be divided into fractions.

Algorithm
1. Start.
2. Accept the weight and profit of each item.
3. Calculate the profit-to-weight ratio.
4. Sort items in descending order of this ratio.
5. Select items with the highest ratio first.
6. If the complete item fits, add it to the knapsack.
7. Otherwise, take the fraction that fits.
8. Calculate and display the maximum profit.
9. Stop.

Time and Space Complexity

Complexity	Value
Time Complexity	O(n log n)
Space Complexity	O(n)

PR-4
Write a program to solve a 0-1 Knapsack problem using dynamic programming or branch and bound strategy

```
# 0/1 Knapsack using Dynamic Programming
def knapsack(weights, profits, capacity, n):
    dp = [[0 for j in range(capacity + 1)]
          for i in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(1, capacity + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(
                    profits[i - 1] + dp[i - 1][w - weights[i - 1]],
                    dp[i - 1][w]
                )
            else:
                dp[i][w] = dp[i - 1][w]
    print("\nMaximum Profit:", dp[n][capacity])

    # Find selected items
    w = capacity
    selected = []
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i - 1][w]:
            selected.append(i)
            w -= weights[i - 1]
    print("Selected Items:", selected[::-1])

# Main Program
n = int(input("Enter number of items: "))
weights = []
profits = []
```

```
for i in range(n):
    weight = int(input(f"Enter weight of item {i+1}: "))
    profit = int(input(f"Enter profit of item {i+1}: "))
    weights.append(weight)
    profits.append(profit)
capacity = int(input("Enter knapsack capacity: "))
knapsack(weights, profits, capacity, n)
```

1. Theory

The 0/1 Knapsack problem is an optimization problem solved using Dynamic Programming. Each item has a weight and profit. The objective is to maximize total profit without exceeding the knapsack capacity. Each item can either be selected completely or rejected; fractional selection is not allowed.

2. Algorithm

1. Start.
2. Accept the number of items, weights, profits and knapsack capacity.
3. Create a DP table of size (n+1) × (W+1) and initialize it with zeros.
4. Traverse the table using nested loops.
5. If item weight is less than or equal to current capacity, calculate maximum profit by including or excluding the item.
6. Otherwise, copy the previous row's value.
7. Display maximum profit.
8. Trace the DP table to find selected items.
9. Stop.

3. Time Complexity O(n\times W)

4. Space Complexity O(n\times W)

Where n = Number of items and W = Knapsack capacity.

PR-5

Design 8-Queens matrix having first Queen placed. Use backtracking to place remaining Queens to generate the final 8-queen’s matrix.

```
# 8 Queens Problem using Backtracking
N = 8
board = [[0 for _ in range(N)] for _ in range(N)]

# Check whether queen can be placed safely
def is_safe(row, col):
    # Check column
    for i in range(row):
        if board[i][col] == 1:
            return False
    # Check upper-left diagonal
    i, j = row - 1, col - 1
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1
    # Check upper-right diagonal
    i, j = row - 1, col + 1
```

```
    while i >= 0 and j < N:
        if board[i][j] == 1:
            return False
        i -= 1
        j += 1
    return True

# Backtracking function
def solve_queens(row):
    if row == N:
        return True
    for col in range(N):
        if is_safe(row, col):
            board[row][col] = 1
            if solve_queens(row + 1):
                return True
            board[row][col] = 0
    return False


# Main Program
# Place first queen at row 0, column 0
board[0][0] = 1
print("Initial Queen Position:")
for row in board:
    print(row)
if solve_queens(1):
    print("\nFinal 8-Queens Matrix:")
    for row in board:
        print(row)
else:
    print("Solution does not exist.")
```

The 8-Queens problem involves placing 8 queens on an 8×8 chessboard so that no two queens attack each other. Backtracking places queens one by one and checks whether each position is safe. If a position leads to no solution, the algorithm backtracks and tries another position.

Algorithm
1. Start.
2. Create an 8×8 chessboard initialized with zeros.
3. Place the first queen at position (0,0).
4. Check each column of the next row for a safe position.
5. Place a queen if no other queen attacks that position.
6. Recursively move to the next row.
7. If no safe position exists, backtrack and remove the previously placed queen.
8. Repeat until all 8 queens are placed.
9. Display the final chessboard matrix.
10. Stop.

Time Complexity O(N!)
Space Complexity O(N^2)

For this program, \(N=8\).

ML
PR-1
Predict the price of the Uber ride from a given pickup point to the agreed drop-off location.
Perform the following tasks:
1. Pre-process the dataset.
2. Identify outliers.
3. Check the correlation.
4. Implement linear regression and random forest regression models.
5. Evaluate the models and compare their respective scores like R2, RMSE, etc.

Install library pip install pandas numpy matplotlib seaborn scikit-learn
Download dataset from Kaggle Uber Fares Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score


df = pd.read_csv("uber.csv")
print(df.head())
print(df.info())
print(df.isnull().sum())


# Remove unnecessary columns
df = df.drop(['Unnamed: 0', 'key'], axis=1, errors='ignore')
# Remove missing values
df = df.dropna()
# Convert pickup datetime
df['pickup_datetime'] = pd.to_datetime(
    df['pickup_datetime'], errors='coerce', utc=True
)
df = df.dropna(subset=['pickup_datetime'])
# Extract date and time features
df['hour'] = df['pickup_datetime'].dt.hour
df['day'] = df['pickup_datetime'].dt.day
df['month'] = df['pickup_datetime'].dt.month
df = df.drop('pickup_datetime', axis=1)
# Remove invalid values
df = df[
    (df['fare_amount'] > 0) &
    (df['passenger_count'] > 0) &
    (df['passenger_count'] <= 6) &
    df['pickup_longitude'].between(-180, 180) &
    df['dropoff_longitude'].between(-180, 180) &
    df['pickup_latitude'].between(-90, 90) &
    df['dropoff_latitude'].between(-90, 90)
]
print(df.head())
```

```
# Boxplot for detecting outliers
plt.figure(figsize=(8, 4))
sns.boxplot(x=df['fare_amount'])
plt.title("Fare Amount Outliers")
plt.show()
# Remove outliers using IQR
Q1 = df['fare_amount'].quantile(0.25)
Q3 = df['fare_amount'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
df = df[
    (df['fare_amount'] >= lower) &
    (df['fare_amount'] <= upper)
]
print("Dataset after removing outliers:", df.shape)


plt.figure(figsize=(10, 6))
sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    cmap='coolwarm'
)
plt.title("Correlation Heatmap")
plt.show()


X = df.drop('fare_amount', axis=1)
y = df['fare_amount']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)


lr = LinearRegression()
lr.fit(X_train, y_train)
lr_pred = lr.predict(X_test)
print("Linear Regression Results")
print("R2 Score:", r2_score(y_test, lr_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, lr_pred)))
print("MAE:", mean_absolute_error(y_test, lr_pred))


rf = RandomForestRegressor(
    n_estimators=50,
    random_state=42,
    n_jobs=-1
)
rf.fit(X_train, y_train)
```

```
rf_pred = rf.predict(X_test)
print("Random Forest Regression Results")
print("R2 Score:", r2_score(y_test, rf_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, rf_pred)))
print("MAE:", mean_absolute_error(y_test, rf_pred))


comparison = pd.DataFrame({
    'Model': ['Linear Regression', 'Random Forest'],
    'R2 Score': [
        r2_score(y_test, lr_pred),
        r2_score(y_test, rf_pred)
    ],
    'RMSE': [
        np.sqrt(mean_squared_error(y_test, lr_pred)),
        np.sqrt(mean_squared_error(y_test, rf_pred))
    ],
    'MAE': [
        mean_absolute_error(y_test, lr_pred),
        mean_absolute_error(y_test, rf_pred)
    ]
})
print(comparison)


sample = X_test.iloc[[0]]
prediction = rf.predict(sample)
print("Predicted Uber Fare:", round(prediction[0], 2))
```

Uber fare prediction is a regression problem used to estimate ride prices based on pickup and drop-off locations, passenger count and other features. Linear Regression and Random Forest Regression are implemented to predict fares. Their performance is compared using R^2 score, RMSE and MAE.

Algorithm

1. Start.
2. Import required Python libraries.
3. Load the Uber dataset.
4. Preprocess data by handling missing values and removing unnecessary columns.
5. Identify and remove outliers using the IQR method.
6. Perform correlation analysis using a heatmap.
7. Separate independent and dependent variables.
8. Split the dataset into training and testing sets.
9. Train Linear Regression and Random Forest Regression models.
10. Predict fare prices using both models.
11. Evaluate and compare models using R^2, RMSE and MAE.
12. Stop.

Time and Space Complexity

Operation	Complexity
Data preprocessing	O(n)
Correlation	$O(n \times m^2)$
Linear Regression training	Approximately $O(n \times m^2 + m^3)$
Random Forest training	Approximately $O(T \times n \times m \times \log n)$

Space Complexity O(n × m + T × n)

Where n = number of records, m = number of features, and T = number of trees.

PR-2
Implement K-Means clustering/ hierarchical clustering on sales_data_sample.csv dataset. Determine the number of clusters using the elbow method.

Install library pip install pandas numpy matplotlib seaborn scikit-learn scipy
Dataset from Kaggle : Sample Sales Data

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster


df = pd.read_csv("sales_data_sample.csv", encoding="latin1")
print(df.head())
print(df.shape)
print(df.info())


# Select numerical columns
data = df.select_dtypes(include=[np.number])
# Handle missing values
data = data.fillna(data.median())
# Remove constant columns
data = data.loc[:, data.nunique() > 1]
# Standardize data
scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)
print("Preprocessed Data:")
print(scaled_data[:5])


wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=42, n_init=10)
    kmeans.fit(scaled_data)
    wcss.append(kmeans.inertia_)
plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.grid()
plt.show()
```

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df['KMeans_Cluster'] = kmeans.fit_predict(scaled_data)
print("K-Means Cluster Results:")
print(df['KMeans_Cluster'].value_counts())


# Create linkage matrix
linked = linkage(scaled_data, method='ward')
# Display dendrogram
plt.figure(figsize=(10, 5))
dendrogram(linked)
plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()


# Assign 3 clusters
df['Hierarchical_Cluster'] = fcluster(
   linked, t=3, criterion='maxclust'
)
print("Hierarchical Cluster Results:")
print(df['Hierarchical_Cluster'].value_counts())


plt.figure(figsize=(8, 5))
plt.scatter(
   scaled_data[:, 0],
   scaled_data[:, 1],
   c=df['KMeans_Cluster'],
   cmap='viridis'
)
plt.title("K-Means Clustering")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

K-Means and Hierarchical Clustering are unsupervised machine learning algorithms used to group similar data points. K-Means divides data into a predefined number of clusters using centroids, while Hierarchical Clustering creates a tree-like structure of clusters. The Elbow Method helps determine the optimal number of clusters by analyzing WCSS values.

Algorithm

1. Start.
2. Import required libraries.
3. Load sales_data_sample.csv.
4. Select numerical features and handle missing values.
5. Standardize the dataset.
6. Apply the Elbow Method by calculating WCSS for different cluster counts.
7. Select the optimal number of clusters.
8. Apply K-Means clustering.
9. Apply Hierarchical Clustering using Ward's method.
10. Plot the dendrogram and visualize clusters.
11. Display cluster assignments.

12. Stop.

Time and Space Complexity
Let n = number of records, k = number of clusters, m = number of features, and I = K-Means iterations.

Algorithm	Time Complexity	Space Complexity
K-Means	$O(n \times k \times m \times I)$	$O(n \times m)$
Hierarchical	$O(n^2 \log n)$	$O(n^2)$
Elbow Method	Depends on K-Means runs	$O(n \times m)$

PR-3
Implement the K-Nearest Neighbors algorithm on the diabetes.csv dataset. Compute confusion matrix, accuracy, error rate, precision and recall on the given dataset.

Install libraries pip install pandas numpy matplotlib seaborn scikit-learn
Dataset from Kaggle: Pima Indians Diabetes Database

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score
)


df = pd.read_csv("diabetes.csv")
print(df.head())
print(df.shape)
print(df.isnull().sum())


# Replace invalid zero values with median
cols = ['Glucose', 'BloodPressure', 'SkinThickness',
        'Insulin', 'BMI']
for col in cols:
    df[col] = df[col].replace(0, df[col].median())
# Separate features and target
X = df.drop('Outcome', axis=1)
y = df['Outcome']
# Feature scaling
scaler = StandardScaler()
X = scaler.fit_transform(X)


X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
```

```
)


knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
print("Predicted Values:")
print(y_pred)


cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
error_rate = 1 - accuracy
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
print("Confusion Matrix:\n", cm)
print("Accuracy:", accuracy)
print("Error Rate:", error_rate)
print("Precision:", precision)
print("Recall:", recall)


import matplotlib.pyplot as plt
import seaborn as sns
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted Values")
plt.ylabel("Actual Values")
plt.title("KNN Confusion Matrix")
plt.show()
```

K-Nearest Neighbors (KNN) is a supervised machine learning classification algorithm. It classifies a new data point based on the majority class of its K nearest neighbors. In this practical, KNN is used to predict whether a person has diabetes based on medical attributes.

Algorithm
1. Start.
2. Import required libraries.
3. Load the diabetes.csv dataset.
4. Handle invalid zero and missing values.
5. Separate independent features and target variable.
6. Apply feature scaling using StandardScaler.
7. Split the dataset into training and testing sets.
8. Initialize KNN with K = 5.
9. Train the model and predict test data.
10. Calculate confusion matrix, accuracy, error rate, precision and recall.
11. Display the results.
12. Stop.

Time Complexity
Let n = training samples, m = number of features and k = neighbors.
- Training: O(1) for basic KNN (stores training data).
- Prediction: O(n × m) per query.
- Overall prediction: O(q × n × m), where q is test samples.

Space Complexity O(n\times m)

PR-4
Given a bank customer, build a neural network-based classifier that can determine whether they will leave or not in the next 6 months. Dataset Description: The case study is from an open-source dataset from Kaggle. The dataset contains 10,000 sample points with 14 distinct features such as Customer ID, Credit Score, Geography, Gender, Age, Tenure, Balance, etc.
Perform the following steps:
1. Read the dataset.
2. Distinguish the feature and target set and divide the data set into
training and test sets.
3. Normalize the train and test data.
4. Initialize and build the model. Identify the points of improvement
and implement the same.
5. Print the accuracy score and confusion matrix (5 points).

Install libraries pip install pandas numpy matplotlib seaborn scikit-learn tensorflow
Dataset from Kaggle: Churn Modelling

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import accuracy_score, confusion_matrix
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping


df = pd.read_csv("Churn_Modelling.csv")
print(df.head())
print(df.shape)
print(df.isnull().sum())


# Remove unnecessary columns
df = df.drop(['RowNumber', 'CustomerId', 'Surname'], axis=1)
# Separate features and target
X = df.drop('Exited', axis=1)
y = df['Exited']
# Convert categorical columns into numerical values
X = pd.get_dummies(X, columns=['Geography', 'Gender'], dtype=int)
print(X.head())


X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
```

```
)


scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)


from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
model = Sequential()
model.add(Input(shape=(X_train.shape[1],)))
model.add(Dense(64, activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
model.summary()


# Add Dropout to reduce overfitting
model = Sequential()
model.add(Dense(64, activation='relu',
                input_shape=(X_train.shape[1],)))
model.add(Dropout(0.3))
model.add(Dense(32, activation='relu'))
model.add(Dropout(0.2))
model.add(Dense(1, activation='sigmoid'))
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
# Handle class imbalance
weights = compute_class_weight(
    class_weight='balanced',
    classes=np.array([0, 1]),
    y=y_train
)
class_weights = {0: weights[0], 1: weights[1]}
# Early stopping
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)
history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2,
```

```
    callbacks=[early_stop],
    class_weight=class_weights,
    verbose=1
)


y_pred = (model.predict(X_test) > 0.5).astype(int)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy * 100, "%")
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)


plt.figure(figsize=(6, 4))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Bank Customer Churn Confusion Matrix")
plt.show()
```

Artificial Neural Network (ANN) is a deep learning technique used for classification and prediction. In this practical, ANN predicts whether a bank customer will leave or stay within the next six months. The model uses customer details as input and predicts the output using hidden layers and an activation function.

4. Algorithm
1. Start.
2. Import required libraries.
3. Load the bank customer dataset.
4. Remove unnecessary columns and handle categorical data.
5. Separate independent features and target variable.
6. Split the dataset into training and testing sets.
7. Normalize the training and testing data using StandardScaler.
8. Build an ANN model using input, hidden and output layers.
9. Improve the model using Dropout, Early Stopping and class weights.
10. Train the model using training data.
11. Predict customer churn on test data.
12. Calculate accuracy and display the confusion matrix.
13. Stop.

5. Time and Space Complexity

Complexity	Value
Time Complexity	O(E × N × W)
Space Complexity	O(W)

Where:
- E = Number of epochs
- N = Number of training samples

- W = Number of network parameters

PR-5
Classify the email using the binary classification method. Email Spam detection has two states:
a) Normal State – Not Spam
b) Abnormal State – Spam.
Use K-Nearest Neighbors and Support Vector Machine for classification. Analyze their performance

Install libraries pip install pandas numpy matplotlib seaborn scikit-learn
Dataset from Kaggle : SMS Spam Collection Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)


df = pd.read_csv("spam.csv", encoding='latin1')
print(df.head())
print(df.shape)


# Select required columns
df = df.iloc[:, :2]
df.columns = ['label', 'message']
# Remove missing values
df = df.dropna()
# Convert labels into numerical values
df['label'] = df['label'].map({
    'ham': 0,
    'spam': 1
})
# Separate features and target
X = df['message']
y = df['label']


tfidf = TfidfVectorizer(stop_words='english')
X = tfidf.fit_transform(X)


X_train, X_test, y_train, y_test = train_test_split(
    X, y,
```

```
    test_size=0.2,
    random_state=42,
    stratify=y
)


knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
knn_pred = knn.predict(X_test)
print("KNN Accuracy:", accuracy_score(y_test, knn_pred))
print("\nKNN Confusion Matrix:")
print(confusion_matrix(y_test, knn_pred))
print("\nKNN Classification Report:")
print(classification_report(y_test, knn_pred))



svm = SVC(kernel='linear')
svm.fit(X_train, y_train)
svm_pred = svm.predict(X_test)
print("SVM Accuracy:", accuracy_score(y_test, svm_pred))
print("\nSVM Confusion Matrix:")
print(confusion_matrix(y_test, svm_pred))
print("\nSVM Classification Report:")
print(classification_report(y_test, svm_pred))



results = pd.DataFrame({
    'Model': ['KNN', 'SVM'],
    'Accuracy': [
        accuracy_score(y_test, knn_pred),
        accuracy_score(y_test, svm_pred)
    ]
})
print(results)
results.plot(
    x='Model',
    y='Accuracy',
    kind='bar',
    legend=False,
    title='KNN vs SVM Accuracy'
)
plt.ylabel("Accuracy")
plt.ylim(0, 1)
plt.show()
```

Email Spam Classification is a binary classification problem used to identify whether an email is spam or normal. K-Nearest Neighbors (KNN) classifies emails based on their nearest data points, while Support Vector Machine (SVM) separates spam and normal emails using an optimal decision boundary. Their performance is compared using accuracy and confusion matrix.

4. Algorithm

1. Start.
2. Import required libraries.
3. Load the email spam dataset.
4. Remove missing values and preprocess the data.
5. Convert spam and normal labels into numerical values.
6. Convert email text into numerical features using TF-IDF.
7. Split the dataset into training and testing sets.
8. Train the KNN classification model.
9. Train the SVM classification model.
10. Predict email categories using both models.
11. Calculate accuracy and confusion matrix.
12. Compare the performance of both models.
13. Stop.

5. Time and Space Complexity
Let n = number of emails and m = number of features.

Algorithm	Time Complexity
KNN Prediction	O(n × m) per email
SVM Training	Depends on dataset and kernel; linear SVM typically scales approximately with n × m
TF-IDF	Depends on total text size

Space Complexity: O(n × m) for the feature matrix, with additional model storage

BCT
PR-1
Write a smart contract on a test network, for Bank account of a
customer for following
operations:
 Deposit money
 Withdraw Money
 Show balance

Open Remix IDE
select files on left corner
Select create -> New File
Rename it and save as **filename.sol**
Click compile
then travesre to deploy(left side panel)
In Deploy tab select Deploy button
in right side check functionlity

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract BankAccount {
    mapping(address => uint256) private balances;

    event Deposited(address indexed customer, uint256 amount);
    event Withdrawn(address indexed customer, uint256 amount);

    // 1. Deposit Money by typing amount directly
    function deposit(uint256 _amount) public {
        require(_amount > 0, "Deposit amount must be greater than zero");
```

```
        balances[msg.sender] += _amount;
        emit Deposited(msg.sender, _amount);
    }

    // 2. Withdraw Money by typing amount directly
    function withdraw(uint256 _amount) public {
        require(_amount > 0, "Withdrawal amount must be greater than zero");
        require(balances[msg.sender] >= _amount, "Insufficient balance");

        balances[msg.sender] -= _amount;
        emit Withdrawn(msg.sender, _amount);
    }

    // 3. Show Balance
    function getBalance() public view returns (uint256) {
        return balances[msg.sender];
    }
}
```

A smart contract is a self-executing program deployed on a blockchain. This Bank Smart Contract allows customers to deposit money, withdraw money and check their account balance. Solidity is used to implement the contract, and Remix IDE is used to compile and deploy it.

3. Algorithm
1. Start.
2. Create a smart contract named Bank.
3. Declare a mapping to store customer addresses and balances.
4. Create a deposit function to accept Ether and update the balance.
5. Create a withdraw function to check sufficient balance and transfer Ether.
6. Create a showBalance function to display the customer's balance.
7. Compile and deploy the smart contract.
8. Execute deposit, withdraw and balance operations.
9. Stop.

Time and Space Complexity

Operation	Time Complexity
Deposit	O(1)
Withdraw	O(1)
Show Balance	O(1)

Space Complexity: O(n), where n is the number of customer addresses stored in the mapping.

PR-2
Write a program in solidity to create Student data. Use the following constructs:
 Structures
 Arrays
 Fallback
Deploy this as smart contract on Ethereum and Observe the transaction fee and Gas values.

Open Remix IDE
select files on left corner
Select create -> New File
Rename it and save as **filename.sol**

Click compile
then travesre to deploy(left side panel)
In Deploy tab select Deploy button
in right side check functionlity

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract StudentData {
    // 1. Structure to hold student details
    struct Student {
        uint id;
        string name;
        uint age;
        string department;
    }

    // 2. Array to store multiple student records
    Student[] public students;

    // Event to log when a student is added
    event StudentAdded(uint id, string name);
    // Event to log fallback/receive calls
    event LogReceived(address sender, uint amount, string message);

    // Function to add a student to the array
    function addStudent(uint _id, string memory _name, uint _age, string memory _department) public {
        students.push(Student(_id, _name, _age, _department));
        emit StudentAdded(_id, _name);
    }

    // Function to get total number of students
    function getStudentCount() public view returns (uint) {
        return students.length;
    }

    // 3. Receive function to handle pure Ether transfers
    receive() external payable {
        emit LogReceived(msg.sender, msg.value, "Receive function called");
    }

    // 3. Fallback function to handle calls with data or unknown functions
    fallback() external payable {
        emit LogReceived(msg.sender, msg.value, "Fallback function called");
    }
}
```

A smart contract is a self-executing program deployed on the Ethereum blockchain. In this program, structures are used to store student details such as roll number, name, age and course. Arrays store multiple student records, while the fallback function executes when a transaction calls an undefined function or sends data that does not match any function.

3. Algorithm

1. Start.
2. Create a smart contract named Student.
3. Define a structure containing roll number, name, age and course.
4. Declare an array to store student records.
5. Create addStudent() to insert student details into the array.
6. Create getStudent() to retrieve student details using the array index.
7. Create getStudentCount() to display the total number of students.
8. Define a payable fallback function to handle unmatched calls.
9. Compile and deploy the smart contract using Remix IDE.
10. Add and retrieve student records.
11. Observe transaction gas usage and transaction fee.
12. Stop.

4. Time and Space Complexity

Operation	Time Complexity
Add Student	O(1) amortized
Get Student	O(1)
Get Student Count	O(1)
Fallback	O(1)

Space Complexity: O(n), where n is the number of student records stored.